

SOLID Principles — Simplified Guide (with PHP Examples + Interview Questions)

Companion to your KISS, DRY, YAGNI, and Law of Demeter guides — same style: simple explanation, PHP before/after code, bullet-point recap, and interview Q&A for each principle.

SOLID is 5 principles for writing object-oriented code that's easy to maintain and extend:

Letter	Principle
S	Single Responsibility Principle
O	Open/Closed Principle
L	Liskov Substitution Principle
I	Interface Segregation Principle
D	Dependency Inversion Principle

S — Single Responsibility Principle (SRP)

The idea in one line

A class should have **only one reason to change** — i.e., it should do exactly one job.

✗ Violates SRP

php

```

class Invoice {
    public function __construct(
        private float $amount,
        private string $customerEmail
    ) {}

    public function calculateTotal(float $taxRate): float {
        return $this->amount + ($this->amount * $taxRate);
    }

    // Persistence logic mixed into a business/domain class
    public function saveToDatabase(): void {
        $pdo = new PDO('mysql:host=localhost;dbname=shop', 'user', 'pass');
        $stmt = $pdo->prepare('INSERT INTO invoices (amount, email) VALUES (?,
        $stmt->execute([$this->amount, $this->customerEmail]);
    }

    // Notification logic mixed in too
    public function sendEmailToCustomer(): void {
        mail($this->customerEmail, 'Your Invoice', "Total: {$this->amount}");
    }
}

```

`Invoice` now has **three reasons to change**: a change in tax calculation rules, a change in how invoices are stored (e.g., switching to MongoDB), or a change in how emails are sent (e.g., switching to SendGrid). Any of those unrelated changes forces you to touch this same class.

✅ Applying SRP

php

```

class Invoice {
    public function __construct(
        private float $amount,
        private string $customerEmail
    ) {}

    public function calculateTotal(float $taxRate): float {
        return $this->amount + ($this->amount * $taxRate);
    }

    public function getAmount(): float { return $this->amount; }
    public function getCustomerEmail(): string { return $this->customerEmail; }
}

class InvoiceRepository {
    public function save(Invoice $invoice): void {
        $pdo = new PDO('mysql:host=localhost;dbname=shop', 'user', 'pass');
        $stmt = $pdo->prepare('INSERT INTO invoices (amount, email) VALUES (?, ?)');
        $stmt->execute([$invoice->getAmount(), $invoice->getCustomerEmail()]);
    }
}

class InvoiceMailer {
    public function sendInvoiceEmail(Invoice $invoice): void {
        mail($invoice->getCustomerEmail(), 'Your Invoice', "Total: {$invoice->getAmount()}");
    }
}

```

Now each class has **one job**: `Invoice` holds/calculates invoice data, `InvoiceRepository` handles persistence, `InvoiceMailer` handles notification. Changing the database has zero impact on email logic, and vice versa.

Quick recap

- One class = one responsibility = one reason to change
- If you describe a class using "and" ("this class calculates totals **and** saves to DB **and** sends email"), it's probably violating SRP
- Not about "one method per class" — a class can have several methods, as long as they all serve the *same* responsibility
- Benefits: easier to test, easier to understand, changes are isolated

Interview questions — SRP

1. What is the Single Responsibility Principle, and why does it matter?

2. **How do you identify if a class violates SRP?** (Look for multiple unrelated reasons to change; classes named vaguely like `Manager`, `Handler`, `Utility` are common offenders.)
 3. **Does SRP mean a class should only have one method?** (No — one *responsibility*, which can involve multiple related methods.)
 4. **Give an example of a God class/object you've seen or fixed, and how you split it.**
 5. **How does SRP improve testability?** (Smaller, focused classes → simpler unit tests with fewer mocked dependencies.)
-

O — Open/Closed Principle (OCP)

The idea in one line

Classes should be **open for extension, but closed for modification** — you should be able to add new behavior without changing existing, tested code.

✗ Violates OCP

php

```
class DiscountCalculator {
    public function calculate(string $customerType, float $amount): float {
        if ($customerType === 'regular') {
            return $amount * 0.95; // 5% off
        } elseif ($customerType === 'premium') {
            return $amount * 0.90; // 10% off
        } elseif ($customerType === 'vip') {
            return $amount * 0.80; // 20% off
        }
        return $amount;
    }
}
```

Every time a new customer type is added (e.g., "student"), you must **modify** this class and its `if/elseif` chain — risking breaking the existing, already-tested logic for `regular`/`premium`/`vip`.

✓ Applying OCP

php

```

interface DiscountStrategy {
    public function apply(float $amount): float;
}

class RegularDiscount implements DiscountStrategy {
    public function apply(float $amount): float {
        return $amount * 0.95;
    }
}

class PremiumDiscount implements DiscountStrategy {
    public function apply(float $amount): float {
        return $amount * 0.90;
    }
}

class VipDiscount implements DiscountStrategy {
    public function apply(float $amount): float {
        return $amount * 0.80;
    }
}

class DiscountCalculator {
    public function calculate(DiscountStrategy $strategy, float $amount): float {
        return $strategy->apply($amount);
    }
}

// Adding "student" discount later — NO existing class is modified:
class StudentDiscount implements DiscountStrategy {
    public function apply(float $amount): float {
        return $amount * 0.85;
    }
}

```

`DiscountCalculator` never changes again. New discount types are added by **creating new classes**, not editing old ones.

⚠ Note the link to YAGNI here: don't build this `DiscountStrategy` interface on day one for a single discount type. Introduce it once you have a **real, proven need to vary behavior** (e.g., the 2nd or 3rd discount type actually shows up) — see the YAGNI guide's Rule of Three logic.

Quick recap

- Add new behavior via **new classes**, not by editing existing ones

- Typically implemented using interfaces/abstract classes + polymorphism (Strategy pattern is the classic example)
- Protects existing, tested code from regressions when requirements grow
- Don't over-apply on day one — combine with YAGNI judgment

Interview questions — OCP

1. **What is the Open/Closed Principle?**
 2. **How would you refactor a large `if/else` or `switch` statement to follow OCP?** (Strategy pattern / polymorphism.)
 3. **Is OCP always desirable from day one? Why or why not?** (No — ties to YAGNI; premature abstraction adds complexity for hypothetical variants.)
 4. **What design patterns commonly support OCP?** (Strategy, Decorator, Factory, Template Method.)
 5. **Give an example where violating OCP caused a production bug.** (Editing a shared conditional broke an existing, unrelated case.)
-

L — Liskov Substitution Principle (LSP)

The idea in one line

A subclass should be usable **anywhere its parent class is expected**, without breaking the program's correctness.

✗ Violates LSP — the classic Rectangle/Square trap

php

```

class Rectangle {
    protected float $width;
    protected float $height;

    public function setWidth(float $width): void { $this->width = $width; }
    public function setHeight(float $height): void { $this->height = $height; }
    public function getArea(): float { return $this->width * $this->height; }
}

class Square extends Rectangle {
    public function setWidth(float $width): void {
        $this->width = $width;
        $this->height = $width; // forces height to match – breaks Rectangle's
    }
    public function setHeight(float $height): void {
        $this->width = $height;
        $this->height = $height;
    }
}

function testRectangle(Rectangle $rect): void {
    $rect->setWidth(5);
    $rect->setHeight(4);
    // A caller expecting Rectangle behavior assumes area = 5 * 4 = 20
    assert($rect->getArea() === 20.0); // FAILS for Square – area is 16
}

```

`Square` *technically* extends `Rectangle`, but it silently breaks an assumption any caller of `Rectangle` reasonably makes (that width and height can be set independently).

Substituting a `Square` where a `Rectangle` is expected produces wrong behavior — this is exactly what LSP forbids.

✅ Applying LSP

php

```

interface Shape {
    public function getArea(): float;
}

class Rectangle implements Shape {
    public function __construct(private float $width, private float $height) {}
    public function getArea(): float {
        return $this->width * $this->height;
    }
}

class Square implements Shape {
    public function __construct(private float $side) {}
    public function getArea(): float {
        return $this->side * $this->side;
    }
}

function printArea(Shape $shape): void {
    echo $shape->getArea() . "\n"; // works correctly for ANY Shape
}

```

Instead of forcing `Square` to inherit from `Rectangle` (a relationship that looks correct mathematically but breaks behaviorally), both implement a common `Shape` interface with only the guarantee they can *actually* both honor: "I can compute my area."

Quick recap

- Subclass must honor the **behavioral contract** of its parent, not just its structure
- Signs of violation: subclass overrides a method to do nothing (`throw new Exception`), or changes expected behavior/results
- "Is-a" in the real world (a square *is* a rectangle, mathematically) doesn't always mean "is-a" in code, if behavior differs
- Rule of thumb: if substituting the subclass could surprise a caller written against the base type, LSP is violated

Interview questions — LSP

1. What is the Liskov Substitution Principle, in your own words?
2. Explain the Rectangle/Square problem and why it violates LSP.
3. How do you detect an LSP violation during code review? (Overridden methods that throw exceptions, do nothing, or change return semantics/preconditions.)
4. Give a real example (not geometry) where LSP was violated. (E.g., a `ReadOnlyList` extends `List` whose `add()` throws `UnsupportedOperationException` — breaks callers expecting `List` to support `add()`.)

5. **How does LSP relate to interface design?** (Motivates preferring composition/smaller interfaces over forcing inheritance where behavior doesn't truly match.)
-

I — Interface Segregation Principle (ISP)

The idea in one line

Don't force a class to implement methods it doesn't use — prefer **many small, specific interfaces** over one large, general-purpose one.

✗ Violates ISP

```
php

interface Worker {
    public function work(): void;
    public function eat(): void;
    public function sleep(): void;
}

class HumanWorker implements Worker {
    public function work(): void { echo "Working...\n"; }
    public function eat(): void { echo "Eating lunch...\n"; }
    public function sleep(): void { echo "Sleeping...\n"; }
}

class RobotWorker implements Worker {
    public function work(): void { echo "Working 24/7...\n"; }

    // Robots don't eat or sleep, but are FORCED to implement these
    public function eat(): void {
        throw new Exception("Robots don't eat!");
    }
    public function sleep(): void {
        throw new Exception("Robots don't sleep!");
    }
}
```

`RobotWorker` is forced to implement `eat()` and `sleep()` even though neither makes sense for it — resulting in methods that exist only to throw exceptions. Any code calling `eat()` on a generic `Worker` might now unexpectedly blow up if it happens to receive a `RobotWorker`.

✓ Applying ISP

```
php

interface Workable {
    public function work(): void;
}

interface Eatable {
    public function eat(): void;
}

interface Sleepable {
    public function sleep(): void;
}

class HumanWorker implements Workable, Eatable, Sleepable {
    public function work(): void { echo "Working...\n"; }
    public function eat(): void { echo "Eating lunch...\n"; }
    public function sleep(): void { echo "Sleeping...\n"; }
}

class RobotWorker implements Workable {
    public function work(): void { echo "Working 24/7...\n"; }
    // No eat() or sleep() – not forced to implement what doesn't apply
}
```

Each class implements only the interfaces that are actually relevant to it. `RobotWorker` is never forced to fake an `eat()` method.

Quick recap

- Split large "fat" interfaces into smaller, role-specific ones
- A class should never be forced to implement a method with no meaningful behavior (throwing `NotImplementedException` is a red flag)
- Directly improves LSP compliance too — no more surprising exceptions from unsupported operations
- Classic real-world example: printers — a `SimplePrinter` shouldn't be forced to implement `scan()`/`fax()` if it can only print

Interview questions — ISP

1. **What is the Interface Segregation Principle?**
2. **How is ISP different from SRP?** (SRP is about a *class* having one responsibility; ISP is about an *interface* not forcing unrelated methods onto its implementers.)

3. **What's a code smell that signals an ISP violation?** (Methods that throw `UnsupportedOperationException`/empty method bodies in implementing classes.)
 4. **Give a real-world example of ISP** (printer interfaces, or payment gateways where not all support refunds).
 5. **How does ISP affect how you write interfaces during initial design?** (Favor small, cohesive interfaces from the start; combine via multiple interface implementation rather than one large contract.)
-

D — Dependency Inversion Principle (DIP)

The idea in one line

High-level modules shouldn't depend on low-level modules directly — **both should depend on abstractions** (interfaces).

✗ Violates DIP

```
php
class MySQLDatabase {
    public function save(string $data): void {
        echo "Saving '$data' to MySQL\n";
    }
}

class OrderService {
    private MySQLDatabase $database;

    public function __construct() {
        $this->database = new MySQLDatabase(); // hard-coded, concrete dependen
    }

    public function placeOrder(string $orderData): void {
        $this->database->save($orderData);
    }
}
```

`OrderService` (a high-level business class) directly depends on `MySQLDatabase` (a low-level, concrete implementation detail). Switching to PostgreSQL, or writing to a mock database in tests, requires modifying `OrderService` itself.

✓ Applying DIP

```
php
```

```

interface Database {
    public function save(string $data): void;
}

class MySQLDatabase implements Database {
    public function save(string $data): void {
        echo "Saving '$data' to MySQL\n";
    }
}

class PostgreSQLDatabase implements Database {
    public function save(string $data): void {
        echo "Saving '$data' to PostgreSQL\n";
    }
}

class OrderService {
    // depends on the ABSTRACTION, not a concrete class
    public function __construct(private Database $database) {}

    public function placeOrder(string $orderData): void {
        $this->database->save($orderData);
    }
}

// Usage - the concrete choice is decided OUTSIDE OrderService
$service = new OrderService(new MySQLDatabase());
$service->placeOrder('Order #123');

// Swapping databases requires zero changes inside OrderService:
$service2 = new OrderService(new PostgreSQLDatabase());

```

`OrderService` now depends only on the `Database` interface. Both `OrderService` (high-level) and `MySQLDatabase` (low-level) depend on the same abstraction — that's the "inversion": instead of high-level code depending on low-level details, both depend on a contract in between. This also makes unit testing trivial (inject a fake/in-memory `Database` implementation).

Quick recap

- High-level business logic should depend on **interfaces**, not concrete implementations
- Concrete classes are "injected" from outside (this is where **Dependency Injection** comes in — DI is the *technique*, DIP is the *principle* it helps satisfy)
- Makes swapping implementations (real DB ↔ mock DB, one payment gateway ↔ another) trivial and safe

- Don't confuse DIP with just "using interfaces everywhere" — it specifically means **inverting the direction of dependency** from high-level → low-level, to both → abstraction

Interview questions — DIP

1. **What is the Dependency Inversion Principle, and how is it different from Dependency Injection?** (DIP is the design principle; DI is a technique/pattern commonly used to achieve it — e.g., constructor injection.)
 2. **Why should high-level modules not depend on low-level modules directly?** (Couples business logic to implementation details; makes swapping/testing hard.)
 3. **How does DIP improve unit testing?** (You can inject mock/stub implementations of the abstraction instead of a real database/API.)
 4. **What's the difference between DIP and simply "coding to an interface"?** (DIP specifically concerns the *direction* of dependency between high-level and low-level modules; coding to an interface is the mechanism.)
 5. **Give an example of a DIP violation you've fixed in real code.**
-

Combined Interview-Style Questions (all of SOLID)

These are commonly asked as a set in interviews:

1. **Can you explain all 5 SOLID principles with one example each?** (Use the above.)
 2. **Which SOLID principle do violations of the others often lead back to?** — commonly discussed: violating SRP often cascades into OCP violations (a class doing too much becomes hard to extend without modifying it).
 3. **How do SOLID principles relate to design patterns?** (Strategy → OCP/DIP, Decorator → OCP, Adapter → LSP/ISP, Factory → DIP.)
 4. **Can following SOLID too strictly cause problems?** — Yes: over-applying (e.g., an interface for every class with only one implementation) can violate **KISS/YAGNI**. SOLID principles should be applied when there's a real, current need for the flexibility they provide — not speculatively.
 5. **How would you explain SOLID to a junior developer using a real project you've worked on?** — be ready with a concrete story (e.g., "we had a `userManager` god class doing auth, email, and DB logic — we split it per SRP...").
 6. **Which SOLID principle is hardest to apply correctly, and why?** — LSP is often cited, since it depends on behavioral contracts that aren't always obvious from types alone.
 7. **How does SOLID support unit testing?** — SRP → focused tests; DIP → mockable dependencies; ISP → smaller fakes/mocks needed; OCP → new tests for new classes without touching old test suites; LSP → substitutable test doubles behave correctly.
-

How SOLID connects to your earlier principles

Principle	Relationship
-----------	--------------

SRP	A focused class is naturally simpler (KISS) and easier to fully understand
OCP	Should be introduced when a <i>real</i> second/third variant appears — not preemptively (YAGNI)
LSP	A stricter version of "don't surprise your caller" — related to clean abstractions in DIP
ISP	Prevents forcing unrelated behavior into one contract — a sibling to SRP, but for interfaces
DIP	Naturally supports DRY (shared abstraction, one place to change) and testability

Balance reminder: SOLID principles are powerful, but applying all five aggressively on day one for a simple CRUD feature will often violate KISS and YAGNI. Apply them **as complexity/change actually shows up** — same philosophy as your earlier guides.

Quick Reference — All 5

- **S — Single Responsibility:** One class, one reason to change.
- **O — Open/Closed:** Extend via new code, don't modify existing tested code.
- **L — Liskov Substitution:** Subclasses must behave correctly wherever the parent type is expected.
- **I — Interface Segregation:** Many small, specific interfaces beat one large, general one.
- **D — Dependency Inversion:** Depend on abstractions (interfaces), not concrete implementations; inject dependencies from outside.